

TP Temps Réel - Conception d'un ECU

Régulateur de vitesse

Jean-Loup Chrestien de Beauminy Dolovan Akdag

17 juin 2026

Support : ESP32 (dual core) + FreeRTOS, sous ESP-IDF. Code dans le dossier `ecu/`.

1 Choix d'architecture

On donne à chaque tâche une seule responsabilité. Une boucle monolithique qui lirait l'uart, calculerait le pid et émettrait au même endroit serait impossible à isoler en cas de surcharge et tiendrait mal sa période. On découpe donc en quatre tâches qui communiquent uniquement par des primitives FreeRTOS :

- **rx** : lit le bus uart et décode les trames (machine à états octet par octet). seul point d'entrée des données.
- **control** : le pid, réveillé toutes les 100 ms, émet la commande moteur.
- **stats** : la télémétrie, une fois par seconde.
- **failsafe** : la sécurité, endormie, réveillée par un événement (front gpio ou timeout watchdog).

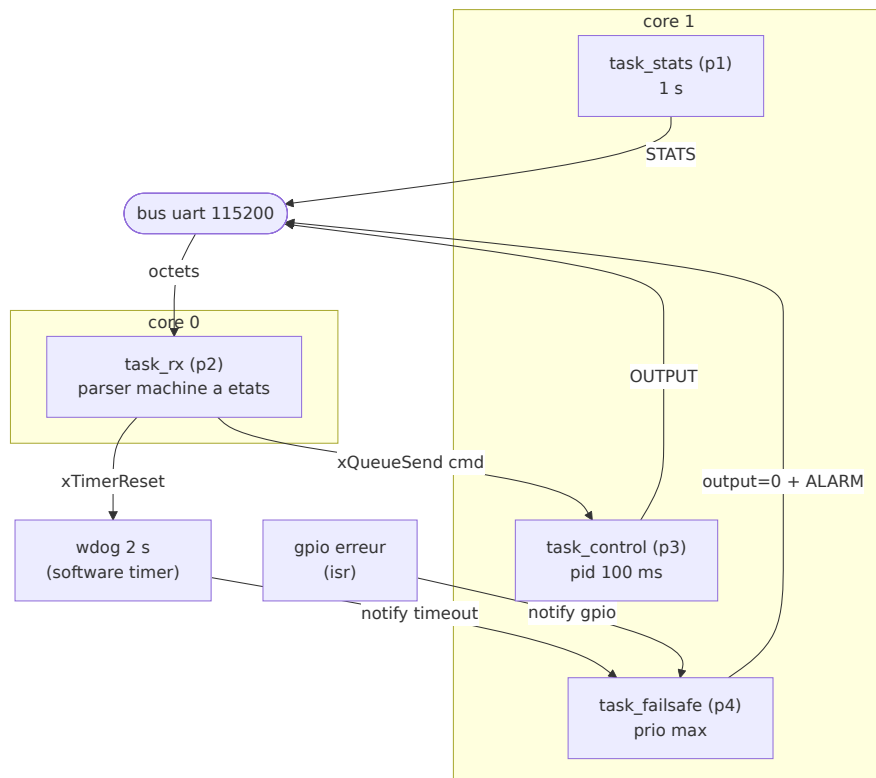
L'isolation repose sur les deux cœurs : rx tourne sur le **core 0**, et tout ce qui est critique (control, failsafe, stats) sur le **core 1**. Un flood uart sature au pire le core 0, mais le pid sur le core 1 reste à l'heure (exigence nf02). rx pousse des couples (**type**, **valeur**) dans une **queue** que control dépile à chaque cycle, donc rx ne touche jamais au pid. Si la queue est pleine on jette la commande et on compte, sans bloquer rx. Le mode et le flag failsafe sont protégés par une section critique (spinlock) car les accès sont trop courts pour justifier un mutex.

2 Tableau des tâches

nom	prio	cœur	périodicité	stack (octets)
failsafe	4	1	événementielle	2048
control	3	1	périodique 100 ms	2560
rx	2	0	bloquante sur uart	3072
stats	1	1	périodique 1000 ms	3072

Priorités de 1 (idle+1) à 4. failsafe au plus haut pour son temps de réponse, stats au plus bas car non critique. Stacks dimensionnées à la louche, vérifiables avec `uxTaskGetStackHighWaterMark`.

3 Diagramme des tâches et événements



tâches, cœurs et flux d'événements

Événements : arrivée d'octets uart (rx decode, push queue, réarme le watchdog) ; tick 100 ms (control dépile, calcule, émet en mode auto) ; tick 1 s (stats émet) ; 2 s sans trame valide (timer expire, notifie failsafe) ; front

gpio d'erreur (isr notifie failsafe) ; `mode_set` (rx remet le mode et efface le flag failsafe, reprise exigence 06).

4 Justification des mécanismes FreeRTOS

- **queue** rx vers control : découple producteur et consommateur, lisse les rafales, envoi non bloquant (timeout 0) pour ne pas ralentir le parser.
- **mutex** sur l'émission uart : trois tâches émettent (control, stats, failsafe), sans protection deux écritures entrelaceraient leurs octets (interdit nf03). On prend un mutex et pas un sémaphore pour l'**héritage de priorité** : si stats (p1) tient le mutex et que failsafe (p4) le demande, stats est remontée à p4 le temps de libérer, pas d'inversion de priorité (nf04).
- **section critique** / **spinlock** pour les compteurs et l'état : accès de quelques cycles, sur dual core il faut un spinlock pour rester cohérent entre les cœurs, plus rapide qu'un mutex.
- **software timer** pour le watchdog rx : one-shot de 2 s réarmé à chaque trame valide, son callback notifie failsafe s'il expire. pas de tâche qui boucle.
- **task notification** pour réveiller failsafe : le plus rapide depuis une isr ou un callback, la valeur transporte la cause (gpio ou timeout).
- **vTaskDelayUntil** pour control et stats : période absolue sans dérive (un vTaskDelay accumulerait le temps de calcul et ferait du jitter).

5 Charge cpu théorique

Tick FreeRTOS réglé à 1000 Hz (1 ms) dans `sdkconfig.defaults` : à 100 Hz la résolution serait de 10 ms, impossible de garantir un jitter sous 5 ms. Les wcet ci-dessous sont des estimations (pas de mesure oscillo).

Core 1 : control (dépile + pid + trame output) estimé 200 us / 100 ms = **0,2 %** ; stats estimé 500 us/s = **0,05 %** ; failsafe négligeable. Total core 1 sous **1 %**.

Core 0 : rx reçoit à 115200 bauds, soit 11520 octets/s, chaque octet passant dans le parser (0,5 us) plus le coût du driver, ce qui reste autour de **1 à 2 %** même en flood continu.

Même sous surcharge uart maximale, le core 1 (la régulation) n'est donc quasiment pas chargé et le pid garde son déterminisme.

6 Priorité de l'uart par rapport au contrôle moteur

Si on traitait l'uart en priorité haute, un flood retarderait le pid et la période de 100 ms ne serait plus tenue. On fait l'inverse : **control (prio 3)**

est au dessus de **rx (prio 2)**, donc un cycle pid préempte le parser. Les deux tâches sont en plus sur des cœurs différents, elles ne se disputent donc pas le cpu. La réception n'est pas sacrifiée pour autant : la queue absorbe les rafales, et la perte éventuelle sous flood extrême est comptée (dropped) sans bloquer. La régulation prime, la réception se dégrade proprement.

7 Choix des priorités et préemption

Ordonnanceur préemptif à priorité fixe. L'ordre suit la criticité temps réel : **failsafe (4)** la sécurité passe avant tout, dès qu'elle est notifiée elle préempte n'importe quoi et force output=0 sous 5 ms sans attendre le prochain cycle pid (l'isr gpio se limite à une notification) ; **control (3)** la régulation, prioritaire sur tout sauf la sécurité ; **rx (2)** important mais ne doit pas perturber control ; **stats (1)** aucune contrainte dure. La préemption joue donc failsafe > control > rx > stats, exactement l'ordre voulu.

8 Monitoring et debug

- **trame stats (0x83)** sur le bus chaque seconde : rx valides, crc erreur, dropped, tx output, unknown (ids inconnus). l'outil pc suit l'ecu en direct.
- **log console** sur l'uart0 (séparé du bus sur uart2) : mode, flag failsafe, compteurs, **heap libre** (`esp_get_free_heap_size`, détecte une fuite) et **jitter max** du pid (`jit_max`, via `esp_timer_get_time`).
- **led** sur le gpio 2 : allumée en failsafe, éteinte à la reprise, debug visuel sans console.
- séparer le bus binaire (uart2) des logs texte (uart0) évite de polluer les trames.

9 Analyse de robustesse

Le script de stress injecte : trames collées, fragmentées, crc faux, ids inconnus, flood. La robustesse repose sur le parser, une machine à états (start, len, data, crc) qui garde son état entre deux lectures uart.

- **fragmentées** : on décode octet par octet en conservant l'état, une trame coupée reprend là où elle s'était arrêtée.
- **collées** : après une trame complète on repart à start, l'octet suivant est aussitôt le début de la trame d'après.
- **crc faux** : on recalcule le xor (len+type+payload), si ça ne colle pas on jette, on compte (crc++) et on resynchronise.
- **len incohérente** : len nul ou trop grand, on rejette et on repart à start, pas de débordement.

- **id inconnu** : trame valide mais type non géré, ignorée, unknown++ (le crc était bon donc pas compté en erreur crc).
- **flood** : le parser tourne sur le core 0 sans toucher au pid (core 1), la queue absorbe et droppe en comptant si elle sature.

Les deux déclencheurs de failsafe sont indépendants : silence du pc (watchdog 2 s) ou front gpio d'erreur (sous 5 ms grâce à la tâche prioritaire). Un `mode_set` auto relance ensuite.

Tests. D'abord un test unitaire de la couche protocole sur pc (construction de trame, crc, même machine à états que le firmware), compilé avec `-Wall -Wextra -Werror` : les six cas (trame propre, collées, fragmentée, crc cassé, id inconnu) se comportent comme prévu. Ensuite le firmware complet (esp-idf v5.4.4) sous **qemu xtensa**, le bus uart2 exposé sur un socket tcp, avec le script de stress fourni : c'est le scénario complet rejoué de bout en bout.

```
$ tail -f /home/jljal/esp/qemu_uart0.log
I (7763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (8763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (9763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (10763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (11763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (12763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (13763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (14763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (15763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (16763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (17763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (18763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (19763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (20763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (21763) ecu: mode=0 fs=1 | rxok=0 crc=0 drop=0 out=1 unk=0 | heap=289312 | jit_max=533us
I (22763) ecu: mode=2 fs=0 | rxok=12 crc=0 drop=0 out=10 unk=0 | heap=289312 | jit_max=533us
I (23763) ecu: mode=2 fs=0 | rxok=22 crc=0 drop=0 out=20 unk=0 | heap=289312 | jit_max=533us
I (24763) ecu: mode=2 fs=0 | rxok=32 crc=0 drop=0 out=30 unk=0 | heap=289312 | jit_max=533us
```

run qemu : passage en mode auto (mode=2), régulation qui démarre (rxok, out montent), drop=0. sous flood unk et crc montent (totaux unk=50, crc=2).

La trace confirme : régulation en mode auto, les 50 trames aléatoires comptées en unk=50 sans planter, les deux trames cassées en crc=2, drop=0 sur tout le run (la queue n'a jamais débordé), failsafe au silence (fs=1, output=0, alarme, le script affiche `verification reussie`) et heap stable. La seule chose que qemu ne valide pas, c'est le timing exact (pas cycle-accurate), donc on l'a mesuré sur carte.

Validation sur cible. Firmware flashé sur un ESP32-D0WD-V3 (dual core, 240 MHz). Boot ok, gpio configurés (led en sortie, entrée d'erreur en pullup sur front descendant), et comme rien n'émet le watchdog déclenche le failsafe à 2,3 s (cause=1). On a instrumenté `task_control` avec `esp_timer_get_time` : à chaque cycle on mesure l'écart à la période de 100 ms et on garde le pire cas. Sur la carte `jit_max` plafonne à 179 us (0,18 ms), largement sous les 5 ms demandés.

```

I (225) heap_init: Initializing. RAM available for dynamic allocation:
I (232) heap_init: At 3FFAE6E0 len 00001920 (6 KiB): DRAM
I (237) heap_init: At 3FFB3230 len 0002C000 (179 KiB): DRAM
I (242) heap_init: At 3FFE0440 len 00003AE0 (14 KiB): 0/IRAM
I (247) heap_init: At 3FFE4350 len 00018C00 (111 KiB): 0/IRAM
I (253) heap_init: At 4008EAF0 len 00011500 (69 KiB): IRAM
I (259) spi_flash: detected chip: generic
I (262) spi_flash: flash io: dio
W (265) spi_flash: Detected size(4096k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
I (278) main_task: Started on CPU0
I (281) main_task: Calling app_main()
I (285) ecu: demarrage ecu
I (287) gpio: GPIO[2] InputEn: 0 OutputEn: 1 OpenDrain: 0 Pullup: 0 Pulldown: 0 Intr:0
I (290) gpio: GPIO[4] InputEn: 1 OutputEn: 0 OpenDrain: 0 Pullup: 1 Pulldown: 0 Intr:2
I (303) main_task: Returned from app_main()
I (1287) ecu: mode=0 fs=0 | rxok=0 crc=0 drop=0 out=0 unk=0 | heap=289312 | jit_max=179us
I (1287) ecu: mode=0 fs=0 | rxok=0 crc=0 drop=0 out=0 unk=0 | heap=289312 | jit_max=179us
W (2503) ecu: failsafe cause=1

```

esp32 réel : démarrage, config gpio, télémétrie avec le jitter mesuré (jit_max=179us) et failsafe watchdog (cause=1)

10 Limitations

- testé sous qemu (stress complet) et sur carte (boot, failsafe watchdog, jitter 0,18 ms). pas pu rejouer le **flood réel** sur la carte : le bus est sur uart2, il aurait fallu un second adaptateur usb-série sur les gpio 16/17. le jitter sous forte charge externe reste à confirmer.
- **wcet et charge cpu** sont des estimations, à confirmer avec `vTaskGetRunTimeStats` et le high water mark.
- **coefficients pid** (kp=1, ki=0.1, kd=0.01) ceux fournis, pas réglés pour un vrai véhicule.
- pas de persistance : au reset on repart en mode off, compteurs à zéro.